

FROM POSSIBILITY

ICT313 Big Data for Software Development

LECTURE 9
Map-Reduce Programming

Big Data



- Extremely large datasets that are hard to deal with using relational databases
  - Storage/ Cost
  - Search/ Performance
  - Analytics and Visualization
- Need for parallel processing on hundreds of machines
  - ETL cannot complete within a reasonable time
  - Beyond 24 hrs. – never catchup

Big Data Challenges

- Dig deeply
- Quickly (Real Time)
- And widely

Types of Big Data

- Structured
- Semi-structured
- Unstructured
  - **Structure Data:** In the form of Tables (Oracle, CyBase, MySQL, Relational DBMS etc).
  - **Semi Structure Data:** Logs, Login events for gmail server.
  - **Un-Structure Data:** The most widely generated data. It is generated every second, it is from the social media, facebook, yoytube. Means; a lot of data is generated at every moment.

Comparison with Existing Solutions

New Age Data Sources
High Volume Sources:
1. Switching devices data.
2. Access point data messages.
3. Call data record due to exponential growth in user base.
Variety of Sources:
1. Image and video feeds from social Networking sites
2. Transaction data.
3. GPS data.
4. Call Center voice feeds.
5. E-mail.
6. SMS.
High Velocity Sources
1. Call Data records.
2. Social networking site conversations.
3. GPS data
4. Call Center- Voice to text feeds.

Data Visualization

A wise investment in your big data future:

- Data visualization is the presentation of data in a pictorial or graphical format.
- It enables decision makers to see analytics presented visually, so they can grasp difficult concepts or identify new patterns.
- With interactive visualization, you can take the concept a step further by using technology to drill down into charts and graphs for more detail, interactively changing what data you see and how it's processed.
- Patterns, trends and correlations that might go undetected in text-based **data** can be exposed and recognized easier with **data visualization** software.

Hadoop (Tool)



- A scalable fault-tolerant grid operating system for data storage and processing
- **Hadoop** is an open source, Java-based programming framework that supports the processing and storage of extremely large data sets in a distributed computing environment.
- It is part of the Apache project sponsored by the Apache Software Foundation

Why use Hadoop?

- **Cheaper**
  - Scales to petabytes or more
- **Faster**
  - Parallel data processing
- **Better**
  - Suitable for particular types of Big Data problems

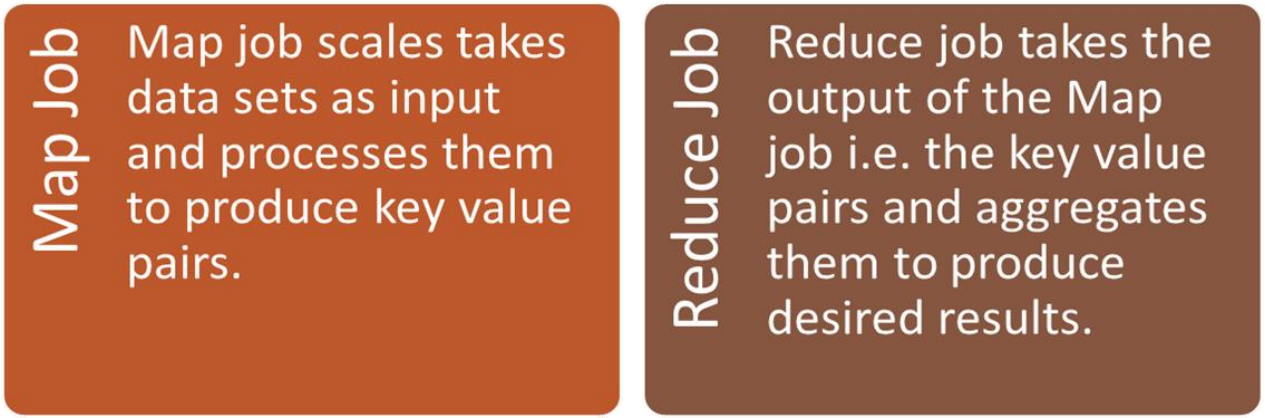
Hadoop Components



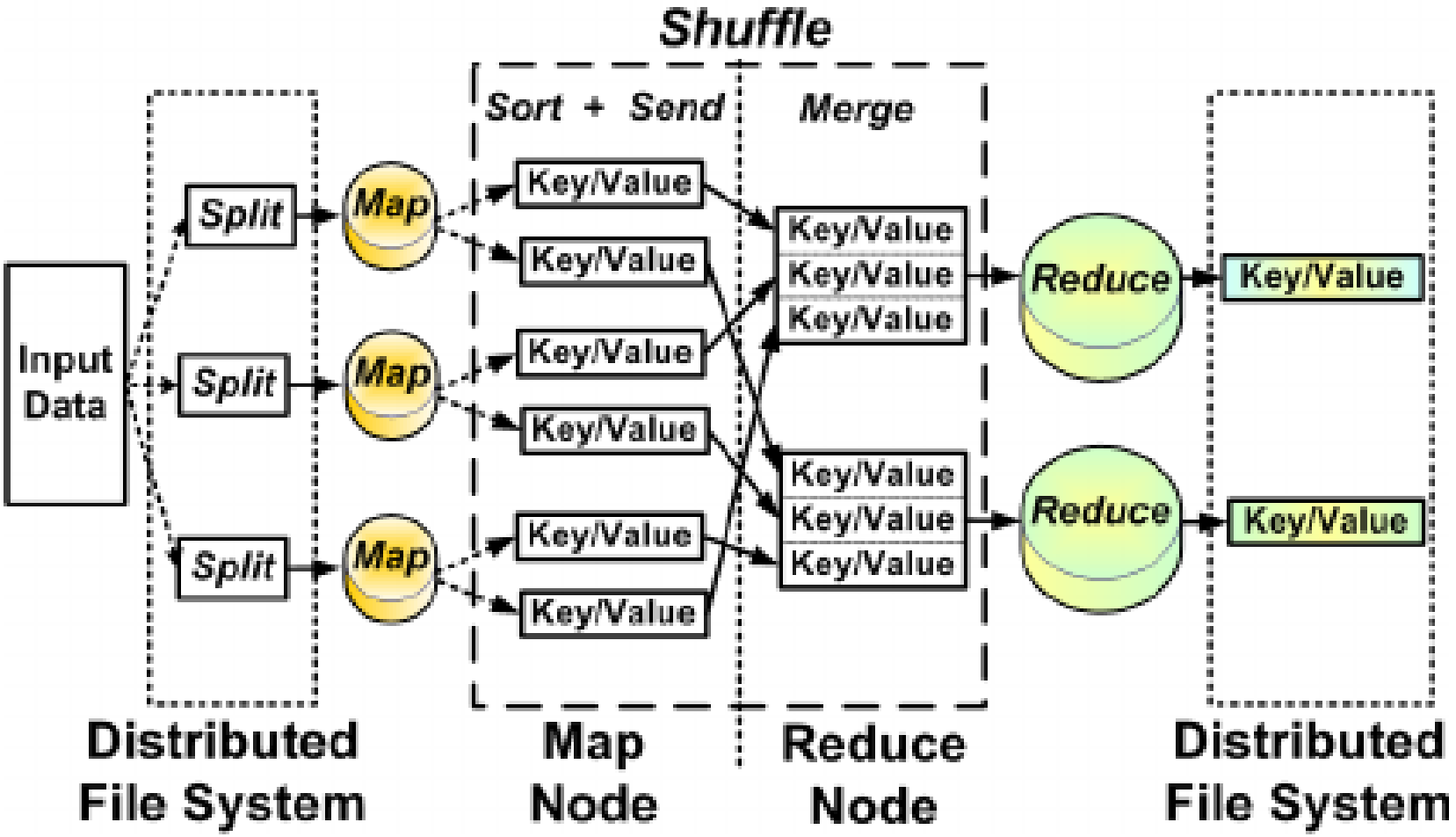
- **Hadoop Distributed File System (HDFS)**
  - Runs on commodity hardware (usually Linux)
  - Horizontally Scalable
- **MapReduce Framework**
  - Parallelized (scalable) processing
  - Manage work distribution and fault Tolerant

MapReduce (Tool)

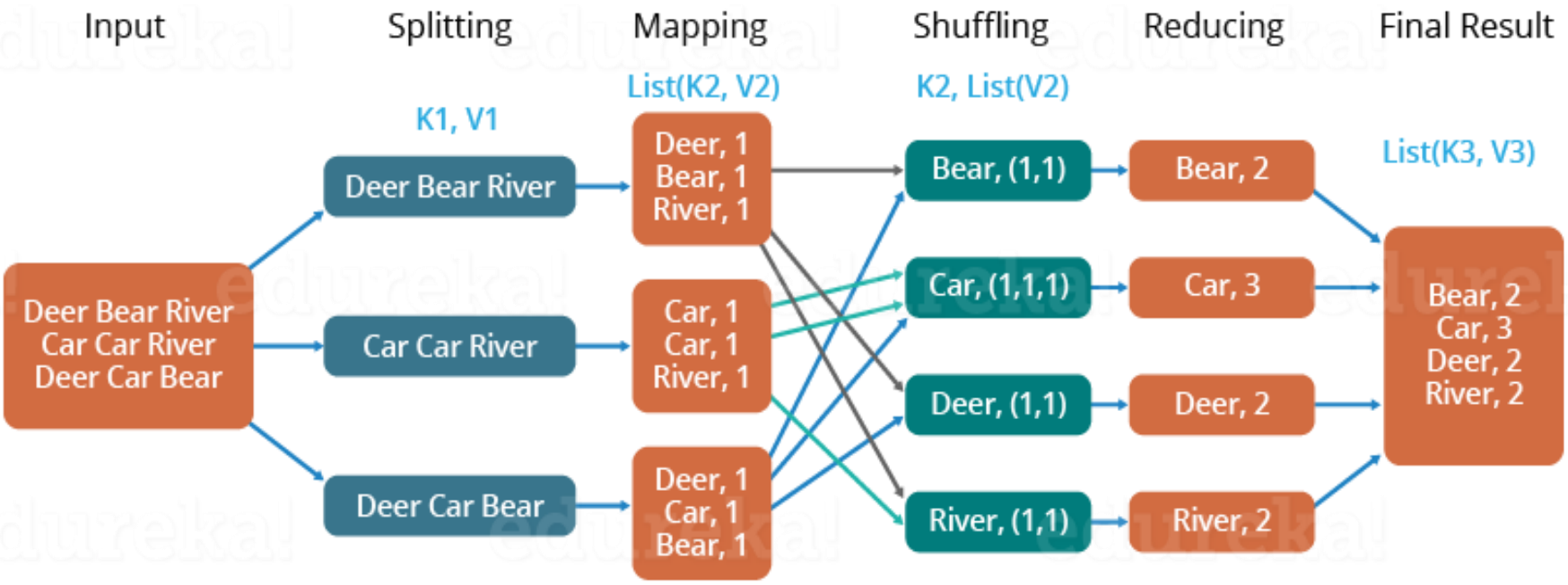
- Hadoop MapReduce is a programming paradigm at the heart of Apache Hadoop for providing massive scalability across hundreds or thousands of Hadoop clusters on commodity hardware.
- The MapReduce model processes large unstructured data sets with a distributed algorithm on a Hadoop cluster.
- It is written in Java and thus gains the advantages of Platform independence.
- The term MapReduce represents two separate and distinct tasks Hadoop programs perform



MapReduce


Shuffle
Sort + Send
Merge
Input Data
Split
Split
Split
Map
Map
Map
Key/Value
Key/Value
Key/Value
Key/Value
Key/Value
Key/Value
Key/Value
Key/Value
Key/Value
Key/Value
Reduce
Reduce
Key/Value
Key/Value
Distributed File System
Map Node
Reduce Node
Distributed File System

MapReduce Example - WordCount



MapReduce

Input key-value pairs	
Key <Object>	Value <Text>
0	This is the first line
23	And this is the second line, and this is all

Mapped key-value pairs	
Key <Text>	Value <IntWritable>
this	1
is	1
the	1
first	1
line	1
and	1
…	…

MapReduce

- reducer function is specified as a class extending the built-in `Reducer<KeyIn, ValueIn, KeyOut, ValueOut> class`

MapReduce

```
public static class MyReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

        public void reduce(Text key, Iterable<IntWritable> values, Context context)
                        throws IOException, InterruptedException {
                int sum = 0;
                IntWritable result = new IntWritable();
                // Summarise the values so far...
                for (IntWritable val : values) {
                        sum += val.get();
                }
                result.set(sum);
                // ... and output a new (word, sum) pair
                context.write(key, result);
        }
}
```

MapReduce

Mapped key-value pairs	
Key <Text>	Value <IntWritable>
this	1
is	1
the	1
first	1
line	1
and	1
this	1
is	1

Mapped key-value pairs for “this”	
Key <Text>	Value <IntWritable>
this	1
this	1

Reduced key-value pairs for “this”	
Key <Text>	Value <IntWritable>
this	1 + 1 = 2

MapReduce

- Constructing MapReduce programs requires a certain skillset in terms of programming
- Tradeoffs in terms of speed, memory consumption, and scalability

Learn MapReduce with Playing Cards

https://www.youtube.com/watch?v=bcjSe0xCHbE